

PrivacyBoost AuthRegistry - Smart Wallet Signature Support Audit



May 21, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	6
Additional Considerations	6
Low Severity	7
L-01 Fallback Verification Failures Collapse Into A Single Error	7
L-02 ERC-7739 Fallback Parser Assumes a 65-Byte Inner Signature	7
L-03 Dual ContentsHash Encodings Accepted For The Same Authorization	8
Notes & Additional Information	8
N-01 ERC-7739 Appendix Parser Only Supports Implicit-Mode Contents Descriptions	8
Conclusion	9
Appendix	10
Issue Classification	10

Summary

Type	DeFi	Total Issues	4 (4 resolved)
Timeline	From 2026-05-13 To 2026-05-14	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	3 (3 resolved)
		Notes & Additional Information	1 (1 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed a focused diff audit of the [AuthRegistry](#) signature-verification changes in the [sunnyside-io/privacy-boost-backend](#) repository, covering commits [2afbe9a](#) (opaque [bytes](#) signatures and EIP-1271 verification) and [c57c1fc](#) (ERC-7739 appendix-aware fallback).

In scope were the following files:

```
contracts/src/  
├─ AuthRegistry.sol  
└─ interfaces/  
    ├─ IAuthRegistry.sol  
    └─ IStructs.sol
```

Update: All resolutions and the final state of the audited codebase mentioned in this report are contained at commit [7bf645e](#).

System Overview

`AuthRegistry` manages BabyJubJub authentication public keys for PrivacyBoost accounts. These keys are used by the protocol's ZK circuits to verify that private actions are authorized by the corresponding BabyJubJub secret key.

The reviewed changes affect the owner authorization path for registry-management actions. The `register`, `rotate`, and `revoke` functions require an owner signature over EIP-712 typed data containing the relevant account, authentication-key, expiry, and nonce fields. These signatures authorize changes to the registry's key set. They do not directly authorize private note spends.

The first change replaces fixed ECDSA tuple verification with opaque `bytes` signatures verified through OpenZeppelin `SignatureChecker`, enabling EIP-1271 smart-wallet owners while preserving legacy overloads. The second change adds a raw-first ERC-7739 fallback for contract wallets that reject the raw `AuthRegistry` EIP-712 digest and instead require Startale and Solady-style wrapped digest verification.

The following review-focus items were examined and produced no findings:

- The legacy `EcdsaSig` overload compatibility.
- The `SignatureChecker` integration for EOAs and ERC-1271 wallets.
- The fallback reachability gating that restricts ERC-7739 verification to contract wallets after raw verification fails.
- The `APP_DOMAIN_SEPARATOR` equality enforcement.
- The storage-layout and upgrade-compatibility impact.

The findings below cover the remaining review-focus items.

Security Model and Trust Assumptions

Account Owner: Authorizes `register`, `rotate`, and `revoke` operations through EIP-712 signatures. For EOA owners, signature validity depends on standard ECDSA verification. For ERC-1271 owners, `AuthRegistry` delegates signature validity to the owner wallet's `isValidSignature` implementation through OpenZeppelin `SignatureChecker`. The protocol relies on smart-wallet owners to implement their signature validation policy correctly, including any ERC-7739 defensive rehashing performed by the wallet. The owner signature path is the trust boundary for deciding which BabyJubJub key is associated with an account.

Additional Considerations

The reviewed EIP-712 owner signatures control registry-management operations only. Private transfers and withdrawals are authorized through BabyJubJub signatures verified inside the ZK circuits. The owner signature layer determines which BabyJubJub keys are present in the registry; it does not itself authorize private actions.

The ERC-7739 fallback is intentionally raw-first: standard EOA and ERC-1271 signatures are checked against the raw `AuthRegistry` EIP-712 digest before the wrapped digest path is attempted. The wrapped path targets Startale and Solady-derived wallets that require appendix-aware wrapped digest verification.

Low Severity

L-01 Fallback Verification Failures Collapse Into A Single Error

The `_verifyOwnerSig` fallback in `AuthRegistry.sol` reverts with `InvalidSignature` for all ERC-7739 failure modes, including malformed appendix length, domain mismatch, contents-description mismatch, contents-hash mismatch, and final wrapped-signature rejection.

This behavior is safe, but makes Startale and Solady integration failures harder to diagnose.

Consider adding granular fallback errors such as `InvalidSignatureLength`, `InvalidERC7739AppDomain`, `InvalidERC7739ContentsDescription`, `InvalidERC7739ContentsHash`, and `InvalidERC7739WrappedSignature`, while keeping `InvalidSignature` for raw verification failures.

Update: Resolved in [pull request #832](#) at commit [d181dba](#).

L-02 ERC-7739 Fallback Parser Assumes a 65-Byte Inner Signature

The `_verifyOwnerSig` bounds check in `AuthRegistry.sol` enforces `66 + n + 65 <= sigLen`, requiring the non-appendix prefix to be at least 65 bytes. ERC-1271 signatures are opaque bytes whose format is defined by the wallet, so this can false-reject valid signatures using shorter or custom encodings.

This is not an accept-invalid path; it only makes the fallback stricter than necessary.

Consider validating only the appendix boundary with `sigLen > 66 + n`, then computing `appStart = sigLen - 66 - n` and letting the wallet determine whether the remaining opaque prefix is valid.

Update: Resolved in [pull request #833](#) at commit [8b06b06](#).

L-03 Dual ContentsHash Encodings Accepted For The Same Authorization

The `_verifyOwnerSig` fallback in `AuthRegistry.sol` accepts both `rawDigest` and `keccak256(abi.encode(CONTENTS_TYPEHASH, structHash))` as valid `contentsHash` values for the same registry action. Both are bound to the same action fields, nonce, chain ID, and registry address, so no direct replay risk is identified.

However, accepting two encodings for the same authorization creates unnecessary ambiguity for signing infrastructure, relayers, monitoring, and auditability.

Consider keeping only the `contentsHash` shape required by the deployed Startale and Solady wallet behavior, or documenting why both are intentionally supported.

Update: Resolved in [pull request #834](#) at commit [9421b2f](#).

Notes & Additional Information

N-01 ERC-7739 Appendix Parser Only Supports Implicit-Mode Contents Descriptions

The `_verifyOwnerSig` function in `AuthRegistry.sol` parses an ERC-7739 appendix from ERC-1271 contract-wallet signatures. The trailing two bytes encode the length of `contentsDescription`, not `contentsType`. The current parser hashes the suffix bytes and compares them to `keccak256("Contents(bytes32 stuff)")`, which only supports the implicit-mode Solady and Startale shape.

This is not an authorization bypass; it can only reject otherwise valid signatures using a different ERC-7739 description shape. However, the names `CONTENTS_TYPEHASH` and `contentsTypeHash` suggest broader ERC-7739 parsing than the implementation provides.

Consider renaming these to `CONTENTS_DESCRIPTION_HASH` and `contentsDescriptionHash`, enforcing `n == 23`, and documenting that the fallback intentionally supports only the Solady and Startale implicit-mode appendix shape.

Update: Resolved in [pull request #835](#) at commit [35e2d87](#).

Conclusion

The reviewed changes extend `AuthRegistry`'s owner authorization path to support opaque `bytes` signatures, EIP-1271 wallets, and a raw-first ERC-7739 fallback for Startale and Solady-derived wallets. The design preserves compatibility with EOAs and standard ERC-1271 wallets while extending support to wallets that require appendix-aware wrapped digest verification. No Critical, High, or Medium severity issues were identified. The findings concern naming clarity, canonicalization of accepted `contentsHash` encodings, an unnecessary inner-signature length bound, and granularity of fallback revert errors.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.